

Computer Vision (All Modules)

Complete Study Material v1

Audit-Ready Study Guide — Exhaustive Coverage with Diagrams, Architectures & Numericals

Contents

1 Module 1: Image Processing Fundamentals

1.1 Introduction to Computer Vision and Image Processing

Computer Vision (CV) is the study of how to enable computers to see, interpret, and understand images and videos, similar to human vision. Making computers understand images like humans understand photos is the central goal of the field.

Image Processing is the technique of manipulating and improving digital images using computer algorithms in order to enhance their quality or extract useful information. It mainly focuses on operations where both the input and the output are images, such as noise removal, contrast enhancement, sharpening, and resizing.

Key Differences:

- **Image Processing:** Focuses on improving or modifying images. Input is an image; output is also an image. Does not try to understand the meaning of the image. Works on pixel values and intensity levels. Used as a preprocessing step before analysis. Examples: noise removal, contrast enhancement, smoothing, sharpening, image compression.
- **Computer Vision:** Focuses on understanding and interpreting images. Input is an image or video; output is information, labels, or decisions. Tries to identify what is present in the image. Works on features, objects, shapes, and patterns. Uses image processing as a foundation. Examples: face recognition, object detection, scene understanding, motion tracking.

Reference book: *Digital Image Processing* – Gonzalez & Woods.

1.2 Digital Images

A **digital image** is a representation of a two-dimensional image as a finite set of digital values, known as picture elements or **pixels**. Each pixel has a specific value that determines its color and brightness, and collectively, these pixels form the complete image.

Formally, a digital image is a 2D function $f(x, y)$ where x, y are spatial coordinates and f is the intensity value. Spatial coordinates tell us the exact position of a pixel in the image.

- x (horizontal coordinate): Tells how far the pixel is from the left side of the image. Increases to the right.
- y (vertical coordinate): Tells how far the pixel is from the top of the image. Increases downward.
- In most digital images, the top-left pixel is $(0, 0)$.

The value $f(x, y)$ tells how bright or dark that pixel is:

- For a grayscale image: $f(x, y) = 0$ represents black, $f(x, y) = 255$ represents white, and values in between represent shades of gray.

Example: $f(100, 50) = 200$ means the pixel at column 100, row 50 has intensity 200 (light/bright gray).

Example (RGB): $f(100, 50) = (255, 0, 0)$ means the pixel at column 100, row 50 is pure red (maximum red, zero green and blue).

1.2.1 Types of Images

1. **Black and White (Binary) Image:** Contains only two possible pixel values — black and white. There are no gray levels.
2. **8-Bit Grayscale Image:** Has 256 intensity levels ranging from 0 to 255. 0 represents black, 255 represents white, and values in between represent shades of gray (e.g., 127 represents medium gray).

3. **16-Bit Image:** Has 65,536 intensity levels (0 to 65,535). Provides much finer intensity detail than an 8-bit image and is commonly used in medical and scientific imaging.

1.2.2 Pixel, Resolution, and Bit Depth

- **Pixel:** The smallest unit of an image.
- **Resolution:** Total number of pixels (e.g., $512 \times 512 = 262,144$ pixels).
- **Bit Depth:** Bits per pixel (8-bit = 256 levels).

Memory Calculation: For an image of size 256×256 with bit depth 8:

$$\text{Memory} = 256 \times 256 \times 8 = 524,288 \text{ bits} = 64 \text{ KB}$$

Higher resolution does not always mean better vision — processing time and memory increase. Medical images often use 16-bit to capture very small intensity variations important in diagnosis.

Dots Per Inch (DPI): A measure of spatial resolution indicating how many dots or pixels are present in one inch of an image. Higher DPI means more detail and better image quality, especially in printing and scanning. DPI describes how many pixels or dots are packed into one inch of physical space, while pixel resolution describes the total number of pixels in an image (width $\times$ height).

Worked Example 1: Image Memory Calculation

Given: Image size = 1024×768 , Bit depth = 24 bits (RGB)

Calculate: Total memory required in MB.

Solution:

1. Total pixels = $1024 \times 768 = 786,432$ pixels
2. Total bits = $786,432 \times 24 = 18,874,368$ bits
3. Total bytes = $18,874,368 / 8 = 2,359,296$ bytes
4. Total KB = $2,359,296 / 1024 = 2,304$ KB
5. Total MB = $2,304 / 1024 \approx 2.25$ MB

Answer: Approximately **2.25 MB**.

1.3 Image Acquisition

Image acquisition is the process of capturing real-world scenes and converting them into digital images. It is the first step in image processing, where a physical scene is sensed and transformed into a digital form.

Why Image Acquisition is Important: All image processing and computer vision tasks depend on image quality. Poor acquisition cannot be fully corrected later. Noise, blur, or low resolution often originate at this stage — “Garbage in = Garbage out.”

1.3.1 Image Acquisition System Components

An image acquisition system includes:

1. **Imaging Sensor:** Captures energy (light, X-ray, infrared, etc.) and converts it into an analog electrical signal.
2. **Sampling:** Converts the continuous spatial signal into discrete pixels.
3. **Quantization:** Converts continuous intensity values into discrete digital values.

The pipeline: Camera sensor captures light (analog signal) → Sampling decides where pixels are placed → Quantization decides pixel intensity values → All together form the image acquisition system.

1.3.2 Three Ways of Image Acquisition

1. **Single Sensor:** Uses one imaging sensor. Example: Old fax machine — scans the scene point by point to form an image.
2. **Sensor Array:** Uses multiple sensors arranged in 1D or 2D. Example: Digital camera or mobile phone camera — captures the entire image at once.
3. **Line Scan Sensor:** Uses a 1D array of sensors. Example: Barcode scanner — uses a 1D sensor array and forms the image line by line using motion.

1.3.3 Complete Acquisition Pipeline

1. **Illumination (Energy) Source:** Provides energy (light, X-ray, infrared, etc.) to illuminate the scene.
2. **Scene Element:** The object or physical scene that reflects or emits energy.
3. **Imaging System:** Captures the reflected energy and forms an image on the image plane using lenses and sensors.
4. **Internal Image Plane:** Continuous (analog) image formed before digitization.
5. **Digitization:** Sampling and quantization convert the analog image into digital form.
6. **Output (Digitized Image):** Final digital image represented as a grid of pixels.

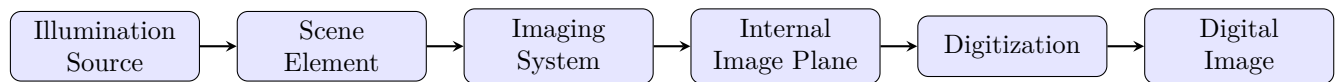


Figure 1: Image Acquisition Pipeline: From illumination source to digital image output.

Worked Example 2: Sampling and Quantization

Given: A continuous image with intensity range $[0, 10]$ volts is to be digitized with 4-bit quantization.

Calculate: Number of intensity levels and the step size.

Solution:

1. Number of intensity levels $= 2^4 = 16$ levels (0 to 15).
2. Step size $= (10 - 0)/16 = 0.625$ volts per level.
3. A continuous value of 3.2 volts would be quantized to level $\lfloor 3.2/0.625 \rfloor = \lfloor 5.12 \rfloor = 5$.
4. Reconstructed value $= 5 \times 0.625 = 3.125$ volts.
5. Quantization error $= 3.2 - 3.125 = 0.075$ volts.

1.4 Image Preprocessing

Image preprocessing consists of operations applied to an acquired image to improve its quality and remove imperfections before further analysis. Preprocessing mainly includes low-level operations such as:

- Noise reduction

- Smoothing and filtering
- Contrast enhancement
- Histogram equalization
- Image sharpening
- Image resizing and normalization
- Image restoration (basic level)

Raw images cannot be directly used for face recognition because they contain noise, lighting variations, and background clutter. Preprocessing is required.

1.4.1 Image Transforms

Image transforms are mathematical techniques that convert an image from the spatial domain (pixel values) into another domain, usually the frequency domain, to make image analysis, enhancement, filtering, and compression easier.

Types of Image Transforms:

1. **Fourier Transform:** Converts an image from spatial domain to frequency domain. Separates the image into low-frequency components (smooth regions) and high-frequency components (edges and details). Used for noise removal, filtering, and image enhancement. (Reference: Gonzalez & Woods, Chapter 4)
2. **Discrete Cosine Transform (DCT):** Represents an image using only cosine functions. Concentrates most image information into a few low-frequency coefficients. Used mainly for image compression (e.g., JPEG).
3. **Wavelet Transform:** Represents an image at multiple resolutions by analyzing both spatial and frequency information. Useful for capturing local image details and edges. Used for image denoising, compression, and feature extraction.

Worked Example 3: DCT Coefficient Energy Compaction

Given: A 2×2 image block with pixel values:

$$I = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}$$

Calculate: The 2D-DCT coefficients.

Solution: For a uniform block, all pixels have the same value (10). The DC coefficient (average) is:

$$F(0,0) = \frac{1}{2}(10 + 10 + 10 + 10) = 20$$

All AC coefficients (high-frequency) are **zero** because there is no variation.

Interpretation: DCT concentrates all energy in the DC coefficient, demonstrating why DCT is excellent for compression — smooth regions need very few coefficients.

1.5 Morphological Image Processing

Morphological image processing is a collection of non-linear operations that process images based on their shape and structure. These operations use a **structuring element** (like a kernel or mask) to probe and modify objects in a binary image (and sometimes grayscale images). Main purpose: to clean shapes, remove noise, fill gaps, separate or connect objects before analysis.

1.5.1 Basic Morphological Operations

1. **Erosion:** Shrinks objects by removing boundary pixels.
 - The structuring element slides over the image.
 - A pixel remains white (1) only if all pixels under the structuring element are white.
 - If any pixel under the structuring element is black, the center pixel becomes black.
 - **Rule:** If ALL pixels under the kernel are 1, center pixel stays 1. If ANY pixel is 0, center pixel becomes 0.
 - **Effects:** Objects become thinner; small noise is removed; narrow connections may break.
 - **Use cases:** Remove small white noise; separate touching objects.
2. **Dilation:** Expands objects by adding pixels to their boundaries.
 - The structuring element slides over the image.
 - If any pixel under the structuring element is white, the center pixel becomes white.
 - **Effects:** Objects become thicker; small holes are filled; close objects may merge.
 - **Use cases:** Fill small gaps; connect broken parts of objects.
3. **Opening:** Erosion followed by Dilation.
 - First erosion removes small noise; then dilation restores the main object size.
 - **Effects:** Removes small objects; smooths object boundaries; preserves shape of larger objects.
 - **Use cases:** Remove salt noise; remove thin protrusions.
4. **Closing:** Dilation followed by Erosion.
 - First dilation fills gaps and holes; then erosion restores object shape.
 - **Effects:** Fills small holes; closes narrow breaks; smooths boundaries.
 - **Use cases:** Fill gaps in characters or objects; repair broken contours.
5. **Hit-or-Miss Transformation:** Detects specific shapes or patterns.
 - Uses two structuring elements: one matches foreground (object), one matches background.
 - A pixel is marked only if both conditions match exactly.
 - **Use cases:** Shape detection; symbol or pattern recognition.

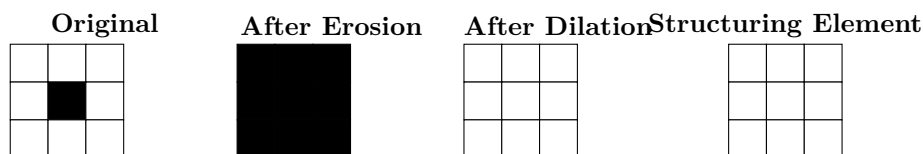


Figure 2: Morphological operations concept: Original binary object, after erosion (shrinks), after dilation (expands), and the structuring element (3×3 kernel).

1.5.2 Morphological Algorithms

- **Boundary Extraction:** Extracts only the boundary (outline) of an object. Obtained by subtracting the eroded image from the original image. Helps in shape analysis and object localization.

- **Region Filling:** Fills holes or enclosed regions inside an object. Starts from a seed point inside the region and expands until the boundary is reached. Used to remove holes in binary objects.
- **Connected Components:** Identifies and labels separate objects in a binary image. Pixels are grouped based on connectivity (4-connected or 8-connected). Used for object counting and region-based analysis.
- **Thinning:** Reduces objects to one-pixel-wide lines. Preserves the overall structure and connectivity. Used in handwriting and fingerprint analysis.
- **Skeletonization:** Produces the medial axis (skeleton) of an object. Represents shape using minimum pixels while preserving topology. Useful for shape recognition and matching.

Easy Recall:

- Boundary = outline
- Region filling = fill holes
- Connected components = separate objects
- Thinning = make thin
- Skeleton = shape backbone

Worked Example 4: Morphological Opening and Closing

Given: A binary image contains a large square object with small white noise dots (salt noise) scattered around it, and a small hole inside the square.

Task: Remove the salt noise and fill the hole.

Solution:

1. **Step 1 – Opening:** Apply erosion followed by dilation with a 3×3 structuring element.
 - Erosion removes the small noise dots (they are smaller than the structuring element).
 - Dilation restores the original size of the square.
 - Result: Square is clean, noise is gone, but the hole remains.
2. **Step 2 – Closing:** Apply dilation followed by erosion with a 3×3 structuring element.
 - Dilation fills the hole inside the square.
 - Erosion restores the square to its original size.
 - Result: Square is clean with no hole.

Final Result: Opening removes noise; Closing fills holes. Both operations preserve the main object.

1.6 Preprocessing Challenges and Solutions

1. **Clutter:** Background contains unnecessary objects that confuse the target.
 - **Solutions:** Image segmentation, morphological opening, background subtraction, ROI (Region of Interest) cropping.
 - **Example:** Detecting a car when trees, people, and buildings are also present.
2. **Deformation:** Same object appears in different shapes or poses.
 - **Solutions:** Morphological operations, edge detection, shape normalization, skeletoniza-

tion.

- **Example:** Handwritten digit “2” written differently by different people.
3. **Intra-class Variation:** Objects of the same class look different in size, color, texture, or orientation.
 - **Solutions:** Image resizing and normalization, histogram equalization, color space conversion (RGB $\rightarrow$ Gray/HSV).
 - **Example:** Faces with different skin tones, lighting, or expressions.
 4. **Gaussian Noise:** Random intensity fluctuations caused by sensors or low light.
 - **Solutions:** Gaussian blur, median filtering, bilateral filtering.
 - **Example:** Grainy medical or satellite images.
 5. **Illumination Variations:** Changes in image appearance caused by lighting, not by the object itself.
 - **Problems:** Pixel intensity changes even when object is same; edges may disappear or false edges appear; ML models may fail due to lighting bias.
 - **Solutions:** Intensity normalization, histogram equalization, adaptive histogram equalization (CLAHE), log/gamma correction, mean-variance normalization.

2 Module 2: Shape Analysis, Edge Detection, and Segmentation

2.1 Introduction to Shape Analysis

Shape analysis is the process of understanding what an object looks like using its outline or its area. Humans do this naturally, but computer vision tries to teach computers the same skill using mathematics.

Why Shape Analysis is Hard for Computers:

1. **Rotation:** For humans, only orientation has changed. For a computer, rotation changes pixel positions completely.
2. **Scale:** Near object = more pixels; far object = fewer pixels. Different size = different pixel count.
3. **Noise:** Poor imaging conditions (cheap cameras, low light, rain/fog) introduce noise, breaking true edges and creating false boundaries.
4. **Occlusion:** When objects are partially hidden, humans mentally complete the shape, but computers need mathematical models to infer missing boundaries.

We need shape descriptions that are robust to rotation, scale, and noise. Two main approaches exist:

- **Contour-based (Boundary) Methods:** Chain codes, boundary representation, Fourier descriptors.
- **Region-based Methods:** Area, perimeter, moments, convex hull, skeletons.

2.2 Contour-Based Shape Representation

A **boundary** is the set of pixels that separate the object from the background. A **contour** is the ordered tracing of that boundary, usually represented as a curve. Contours represent the outline (shape) of an object and separate it from the background.

2.2.1 Chain Codes (Freeman Chain Codes)

Imagine walking along the boundary of an object on graph paper. Instead of storing all pixel coordinates, you just record which direction you move next. That direction sequence = **Chain Code**.

4-Direction Chain Code:

- Move right = 0, Move up = 2, Move left = 4, Move down = 6
- Boundary becomes: 0, 0, 2, 2, 4, 4, 6, 6, ...

8-Direction Chain Code: Uses 8 directions (including diagonals):

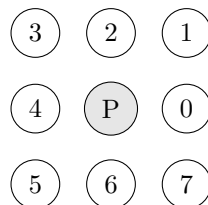


Figure 3: 8-Directional Freeman Chain Code encoding. P = current boundary pixel; numbers = direction of next boundary pixel.

Limitations of Chain Codes:

- **Sensitive to rotation:** Rotating the object changes the direction numbers completely.
- **Sensitive to noise:** Small boundary irregularities create large changes in the code.

- **Depends on starting point:** Different starting pixels give different chain code sequences.
- **Not scale invariant:** Larger or smaller objects produce longer or shorter codes.

Solutions: Normalization using Circular First Difference (CFD) and normalized CFD.

Worked Example 5: Chain Code Extraction

Given: A simple square object traced clockwise from top-left corner.

Trace: Right → Right → Down → Down → Left → Left → Up → Up

8-Direction Chain Code: 0, 0, 6, 6, 4, 4, 2, 2

Normalization: Compute differences between consecutive directions (circular):

- Differences: $(0 - 0) = 0$, $(0 - 6) = -6 \equiv 2 \pmod{8}$, $(6 - 6) = 0$, $(6 - 4) = 2$, $(4 - 4) = 0$, $(4 - 2) = 2$, $(2 - 2) = 0$, $(2 - 0) = 2$
- Normalized Chain Code (CFD): 0, 2, 0, 2, 0, 2, 0, 2

Property: The normalized code is **rotation invariant** — rotating the square gives the same CFD regardless of starting direction.

2.2.2 Fourier Descriptors

Instead of describing shape in the spatial domain, Fourier descriptors describe it in the frequency domain.

Process:

1. Convert the boundary into a complex signal ($x + iy$ coordinates).
2. Apply the Discrete Fourier Transform (DFT).
3. Keep the first P coefficients (low frequencies = overall shape; high frequencies = fine details).

Properties:

- Low frequencies = overall shape; high frequencies = fine details.
- Small P = coarse shape; large P = detailed shape.
- Can be made invariant to translation, rotation, and scale after normalization.

Limitations: Needs closed boundary; loses exact pixel-level detail; not ideal for very irregular shapes.

2.2.3 Geometric Border Representation and B-Splines

Geometric Border Representation: Represents the boundary using geometry (ordered points, straight lines, or curves) instead of pixel directions.

- **Pros:** More precise than chain codes; easy to compute length, angle, curvature.
- **Limitation:** Sensitive to noise and small boundary changes.

B-Spline Representation: Uses smooth polynomial curves (B-splines) to model boundaries, giving a compact and smooth shape description.

- A few control points define the entire smooth shape.
- Boundary does not pass through all points — it is smoother.
- **Advantages:** Smooth; compact representation; stable under small noise.

2.2.4 Shape Invariants

Numerical features of a shape that remain unchanged under rotation, scaling, and translation. Same object looks different to a computer when rotated or resized, so invariants are essential for

reliable recognition.

Examples: Normalized chain codes, compactness, aspect ratio, circularity.

2.3 Region-Based Shape Representation

Instead of describing shapes using only boundaries, region-based methods describe shapes using all pixels inside the object.

2.3.1 Scalar Region Descriptors

Simple numbers used to describe a region (object) in an image:

- **Area:** How big the region is.
- **Perimeter:** Length of its boundary.
- **Centroid:** Center point of the region.
- **Compactness:** How circular or spread out it is.
- **Aspect Ratio:** Width vs height.

Example: Classifying objects (coin vs button) using area and compactness.

2.3.2 Moments

Statistical measures capturing shape and spatial distribution of a region. Moments encode position + spread of pixels.

Zeroth moment (m_{00}): Total mass/area of the region.

First-order moments (m_{10}, m_{01}): Center of mass (centroid).

Second-order moments: Spread of the region (covariance).

Central moments (subtracting centroid) are translation invariant. Normalized moments are scale invariant.

2.3.3 Convex Hull

The smallest convex shape that fully encloses a region. Imagine stretching a rubber band around an object — the shape formed is its convex hull.

Uses: Measuring object solidity, hand gesture analysis.

2.3.4 Region Skeletons

A thin version of the region that preserves shape and topology. Reduces data while keeping important structure.

Uses: Shape recognition, medical image analysis (blood vessels, bones).

2.3.5 Region Decomposition

Dividing a large region into smaller, meaningful sub-regions. Each sub-region is more uniform, making analysis easier.

Example: A face image divided into eyes, nose, and mouth regions.

2.3.6 Region Neighborhood Graph (RNG)

Represents spatial relationships between regions using a graph:

- Each region = node
- Adjacent regions = edges
- Shows which regions are neighbors

Uses: Scene analysis, understanding object relationships.

2.4 Thresholding and Segmentation

2.4.1 Thresholding

Thresholding is a segmentation technique that converts a grayscale image into a binary image by comparing each pixel intensity with a threshold value T :

- If pixel value $\geq T \rightarrow$ object (foreground)
- If pixel value $< T \rightarrow$ background

Basic Principle: Thresholding assumes that object and background have different gray-level intensities. The pixel intensity histogram helps in selecting a threshold (two peaks = one for object, one for background; threshold lies between peaks).

Role of Illumination: Thresholding works best when illumination is uniform. Uneven lighting causes wrong segmentation; shadows cause misclassification.

Solutions for Uneven Illumination:

1. **Illumination Normalization:** Reduces the effect of uneven lighting.
2. **Retinex:** An image processing theory used to reduce illumination effects and enhance object details.
3. **Log Transformation:** Compresses high intensity values; enhances dark regions.
4. **Histogram Equalization:** Improves overall image contrast; spreads gray levels uniformly.
5. **Adaptive (Local) Thresholding:** Computes threshold locally for each region; handles shadows and brightness variations.
6. **Mean-based Local Thresholding:** Threshold is calculated using the mean intensity of a local window.

2.4.2 Global Thresholding (Iterative Method)

Algorithm:

1. Choose an initial threshold T .
2. Segment the image using T .
3. Compute the mean intensity of object and background.
4. Update $T = (\text{Object mean} + \text{Background mean})/2$.
5. Repeat until T does not change.

Worked Example 6: Iterative Global Thresholding

Given: Grayscale pixel values: 15, 20, 25, 30, 180, 190, 200, 210

Initial threshold: $T_0 = 100$

Iteration 1:

- Background (< 100): 15, 20, 25, 30. Mean = $(15 + 20 + 25 + 30)/4 = 22.5$
- Object (≥ 100): 180, 190, 200, 210. Mean = $(180 + 190 + 200 + 210)/4 = 195$
- New $T = (22.5 + 195)/2 = 108.75$

Iteration 2:

- Background (< 108.75): 15, 20, 25, 30. Mean = 22.5
- Object (≥ 108.75): 180, 190, 200, 210. Mean = 195
- New $T = (22.5 + 195)/2 = 108.75$ (no change)

Final Threshold: $T = 108.75$

2.4.3 Otsu's Method

An automatic global thresholding method that chooses the threshold that minimizes intra-class variance (or maximizes inter-class variance).

Intra-class variance: How much pixel values vary within the same class. Small variance $\rightarrow$ pixels are similar $\rightarrow$ good segmentation.

Algorithm:

1. For each possible threshold T : divide pixels into background and foreground.
2. Compute mean and variance of each class.
3. Compute weighted intra-class variance (variance $\times$ number of pixels in that class).
4. Choose T with minimum weighted variance.

Worked Example 7: Otsu's Method

Given: Pixels (grayscale values): [10, 12, 14, 50, 52, 55]

Test: $T = 20$

Background = [10, 12, 14]; **Foreground** = [50, 52, 55]

Means:

$$\mu_1 = (10 + 12 + 14)/3 = 12, \quad \mu_2 = (50 + 52 + 55)/3 \approx 52.3$$

Variances:

$$\sigma_1^2 = [(10 - 12)^2 + (12 - 12)^2 + (14 - 12)^2]/3 = (4 + 0 + 4)/3 \approx 2.67$$

$$\sigma_2^2 = [(50 - 52.3)^2 + (52 - 52.3)^2 + (55 - 52.3)^2]/3 \approx 4.22$$

Weighted variance: $3 \times 2.67 + 3 \times 4.22 = 20.67$

Testing $T = 13$ gives much larger variance (≈ 1428.8), so $T \approx 20$ is best.

2.5 Region-Based Segmentation

2.5.1 Region Growing

Start from a seed pixel and grow the region by adding similar neighboring pixels.

1. Select seed point.
2. Check neighbors.

3. Add similar pixels.
4. Stop when no more pixels match.

Example: Tumor region detection starting from a seed pixel.

2.5.2 Region Splitting and Merging

Region Splitting: Divide image into smaller regions until uniformity is satisfied.

Region Merging: Merge adjacent regions if they are similar.

Why both?: Imagine an image with forest, water, and urban area. Initially the entire image is not uniform, so split until regions have similar texture. After splitting, many small regions have the same texture, so merge neighboring regions with similar texture.

2.6 Pixel Connectivity

Defines how pixels are connected in an image:

1. **4-connectivity:** Pixels are connected if they share an edge (up, down, left, right).
2. **8-connectivity:** Pixels are connected if they share an edge or corner (includes diagonals).
3. **m-connectivity** (mixed): Diagonal connection is allowed only if there is no 4-connected side path between the pixels. Prevents false merging when objects touch only at corners.

Example:

$$\begin{bmatrix} 1 & 1 & . & . \\ 1 & 1 & . & . \\ . & . & 1 & 1 \\ . & . & 1 & 1 \end{bmatrix}$$

Two blocks of 1s touch only at corners. Under 4-connectivity they are separate; under 8-connectivity they are connected; under m-connectivity they are separate (no side path exists).

2.7 Hough Transform

A technique used to detect geometric shapes (lines, circles, ellipses) in images even if edges are broken or noisy.

Traditional edge detection gives points, not shapes. The Hough Transform converts edge points into parameter space where each point “votes” for possible lines. The line that gets the most votes is detected as the real line.

Example: In a road image, many white lane pixels lie in a line. The Hough Transform detects that line as the lane marking.

Worked Example 8: Hough Transform Line Detection

Given: Three edge points: $(1, 1)$, $(2, 2)$, $(3, 3)$.

Step 1: Convert each point to parameter space using $y = mx + c \rightarrow c = -xm + y$.

- For $(1, 1)$: $c = -m + 1$ (line in parameter space)
- For $(2, 2)$: $c = -2m + 2$ (line in parameter space)
- For $(3, 3)$: $c = -3m + 3$ (line in parameter space)

Step 2: Find intersection in parameter space. All three lines intersect at $m = 1, c = 0$.

Step 3: The detected line is $y = x$.

Interpretation: Even if some edge points are missing, the Hough Transform can still detect the line from the remaining points.

3 Module 3: Object Detection and Convolutional Neural Networks

3.1 Introduction to Object Detection

Object detection identifies what objects are present and where they are located in an image by predicting class labels + bounding box coordinates. Detection = classification + localization.

The Human Brain Detects Objects:

1. **Light and Eye Perception:** Captures visual input.
2. **Feature Detection:** V1 (Primary Visual Cortex) detects edges, lines, and colors; V4 & IT Cortex recognize shapes, textures, patterns.
3. **Object Recognition (Top-Down Processing):** Brain matches visual input with stored memories; uses context and past experiences to interpret objects.

How Do Computers Detect Objects?:

1. **Image Preprocessing:** Convert image into pixels (grayscale/RGB); apply filters (edge detection, etc.).
2. **Feature Extraction:** Deep Learning (CNNs) extracts patterns hierarchically.
3. **Object Detection Methods:** Sliding Window, R-CNN family, YOLO (You Only Look Once), SSD, DETR.

3.2 Types of Object Detection

3.2.1 Two-Stage Object Detection (Region Proposal-Based)

Stage 1 – Region Proposal: The model first generates potential object regions using algorithms like Selective Search or Region Proposal Networks (RPNs).

Stage 2 – Classification & Refinement: Each proposed region is classified and refined to improve accuracy.

Advantages: High accuracy due to separate proposal and classification stages; handles small and overlapping objects better.

Disadvantages: Slower because of the extra proposal step; not ideal for real-time applications.

Examples: R-CNN, Fast R-CNN, Faster R-CNN.

3.2.2 One-Stage Object Detection (End-to-End, No Proposals)

The model directly predicts object locations and classes in one pass without a separate region proposal step.

Advantages: Much faster — ideal for real-time applications; simpler architecture.

Disadvantages: Slightly lower accuracy than two-stage methods, especially for small objects; more false positives due to direct prediction.

Examples: YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), RetinaNet.

3.2.3 Proposal-Free / Transformer-Based Detection

Completely avoids region proposal mechanisms. Uses transformer-based architectures.

Advantages: Most efficient — eliminates proposal generation completely; scales better for large datasets.

Disadvantages: Still evolving — accuracy improvements are ongoing; some methods require huge computational power.

Examples: DETR (DEtection TRansformer), Vision Transformers (ViTs) in object detection.

Worked Example 9: Two-Stage vs One-Stage Comparison

Scenario: A self-driving car must detect pedestrians in real time.

Two-Stage (Faster R-CNN):

- RPN generates 2000 region proposals per image.
- Each proposal is classified and refined.
- Time: 200ms per image (5 FPS).
- Accuracy: High (mAP 75%).
- **Verdict:** Too slow for real-time driving decisions.

One-Stage (YOLOv8):

- Image divided into 19×19 grid; each cell predicts directly.
- Time: 10ms per image (100 FPS).
- Accuracy: Good (mAP 65%).
- **Verdict:** Fast enough for real-time; acceptable accuracy trade-off.

Conclusion: For safety-critical real-time applications, one-stage detectors are preferred despite slightly lower accuracy.

3.3 R-CNN Family

3.3.1 R-CNN (Region-based CNN)

Process:

1. Use Selective Search to generate 2000 region proposals.
2. Warp each region to fixed size (227×227).
3. Pass through CNN to extract features.
4. Use SVM for classification and linear regression for bounding box refinement.

Problem: Each proposal is processed independently — very slow.

3.3.2 Fast R-CNN

Improvement: Instead of processing each proposal separately, the entire image is passed through the CNN once to get a feature map. Region proposals are then projected onto this feature map.

Process:

1. Pass entire image through CNN to get feature map.
2. Project region proposals onto feature map (RoI pooling).
3. Use fully connected layers for classification and bounding box regression.

Speedup: 10x faster than R-CNN.

3.3.3 Faster R-CNN

Improvement: Replaces Selective Search with a **Region Proposal Network (RPN)** that shares convolutional features with the detection network.

Process:

1. CNN backbone extracts features from the entire image.
2. **RPN** slides a small network over the feature map to generate region proposals.

3. RoI pooling extracts fixed-size features from each proposal.
4. Classification + bounding box regression.

Speedup: 250x faster than R-CNN; nearly real-time.

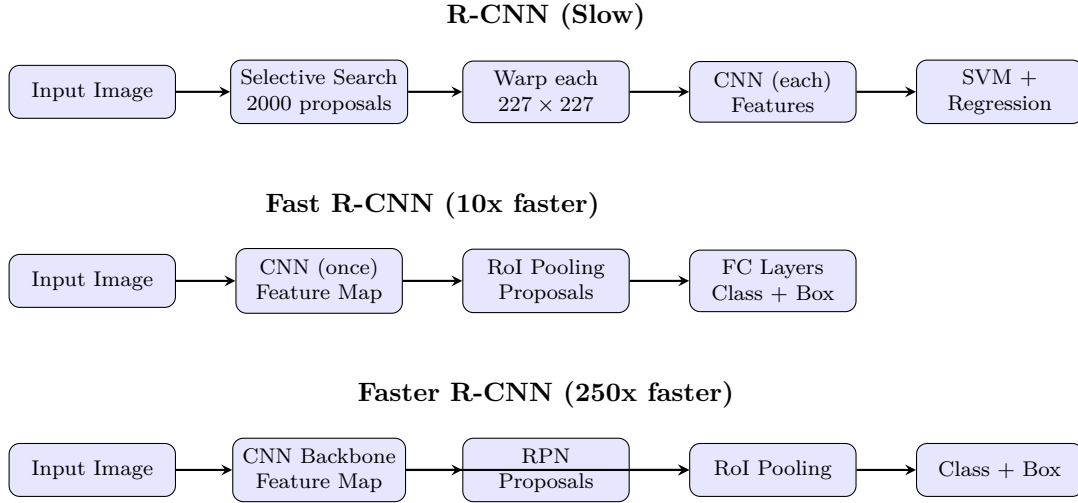


Figure 4: R-CNN evolution: From R-CNN (2000 separate CNN passes) to Fast R-CNN (one CNN pass + RoI pooling) to Faster R-CNN (RPN shares features with detector).

3.4 YOLO (You Only Look Once)

YOLO is a one-stage detector that predicts all bounding boxes in a single forward pass. The input image is resized and divided into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose center lies inside it.

3.4.1 YOLO Grid Prediction

Each grid cell predicts:

- B bounding boxes (each with 5 values: x, y, w, h , confidence)
- Class probabilities for each box

Coordinate system:

- x, y : Box center coordinates relative to grid cell (0–1).
- w, h : Width and height relative to full image (0–1).
- Confidence: Object present + how well the box fits.

Output vector per grid cell: $B \times 5 + N$ (where N = number of classes).

Example: $B = 2, N = 20 \rightarrow$ Each grid cell outputs a 30-length vector.

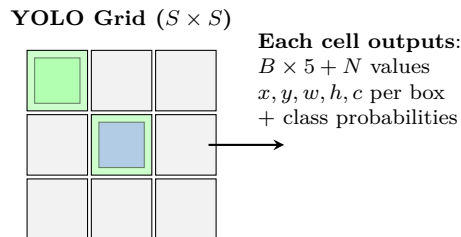


Figure 5: YOLO grid-based detection: Image divided into $S \times S$ grid cells. Cells containing object centers (green) predict bounding boxes; empty cells (gray) predict background.

3.4.2 Anchor Boxes

Problem: In YOLOv1, one grid cell = one object. If two objects have centers in the same cell, one is missed.

Solution: Anchor Boxes. Each grid cell predicts multiple bounding boxes (typically 2 to 9), each with its own confidence and class prediction.

Anchor: Starting box = initial guess (fixed before training).

Bounding Box: Final predicted box = corrected guess (predicted after training).

Example: If a cell has 2 anchors and 2 classes:

- Anchor 1 = 7 values ($x, y, w, h, c + 2$ classes)
- Anchor 2 = 7 values
- Total = 14 values per cell

3.4.3 YOLO Loss Function

The YOLO loss function measures how well the model's predictions match the actual objects. It consists of three parts:

1. Localization Loss (Bounding Box Error):

- Ensures predicted bounding boxes are close to ground truth.
- Uses MSE on x, y, w, h .
- **Square root trick:** $\sqrt{w}$ and $\sqrt{h}$ instead of w and h to balance small and large objects.

2. Confidence Loss (Objectness Error):

- Ensures the model correctly predicts if an object is present.
- Compares predicted confidence with actual presence (1 if object, 0 if not).
- λ_{noobj} : Reduces penalty for empty cells (typically 0.5).

3. Classification Loss (Object Type Error):

- Ensures correct object classification.
- Uses cross-entropy loss between predicted and actual class labels.

$$\text{Total Loss} = \lambda_{\text{coord}} \cdot \text{Localization Loss} + \lambda_{\text{obj}} \cdot \text{Confidence Loss} + \lambda_{\text{class}} \cdot \text{Classification Loss}$$

3.4.4 YOLO Variants

- **YOLOv1 (2015):** First version; divides image into grid; fast but less accurate for small objects.
- **YOLOv2 (2017):** Added anchor boxes, batch normalization, multi-scale training.
- **YOLOv3 (2018):** Feature pyramid for multi-scale detection; Darknet-53 backbone.
- **YOLOv4 (2020):** CSPDarknet-53, mosaic augmentation, self-adversarial training.
- **YOLOv5 (2020):** Smaller and faster; AutoAnchor optimization.
- **YOLOv8 (2023):** Supports detection, segmentation, and tracking in one model.
- **YOLOv11 (2024):** Pose estimation, instance segmentation, greater accuracy with fewer parameters.

3.5 SSD (Single Shot MultiBox Detector)

SSD is a one-stage object detector that uses multiple feature maps at different scales to detect objects of varying sizes.

Architecture:

1. **Input:** 300×300 image.
2. **Backbone (VGG-16):** Extracts features (e.g., $\text{Conv4}_3 \rightarrow 38 \times 38 \times 512$).
3. **Extra Feature Layers:** Produce smaller feature maps (19×19 , 10×10 , 5×5 , etc.).
4. **Prediction Layers:** Each feature map predicts bounding boxes and class probabilities using default/prior boxes.
5. **Post-processing:** Combine predictions from all feature maps; apply NMS.

Key Idea: High-resolution maps detect small objects; low-resolution maps detect large objects.

3.6 Evaluation Metrics

3.6.1 IoU (Intersection over Union)

IoU measures how well the predicted bounding box overlaps with the ground truth box:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{Intersection}}{\text{Union}}$$

Typically, $\text{IoU} > 0.5$ is considered a correct detection.

3.6.2 Precision and Recall

- **TP (True Positive):** Predicted box correctly detects a real object.
- **FP (False Positive):** Predicted box detects something that is not a real object.
- **FN (False Negative):** Real object exists but the model failed to detect it.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

3.6.3 mAP (Mean Average Precision)

1. Sort predictions by confidence score (descending).
2. For each prediction, compute IoU with ground truth.
3. If $\text{IoU} \geq \text{threshold} \rightarrow \text{TP}$; else FP.
4. After each box, calculate cumulative Precision and Recall.
5. **AP** (Average Precision) = area under the Precision–Recall curve for one class.
6. **mAP** = average of AP scores across all classes.

Worked Example 10: IoU, Precision, Recall, and mAP

Given: An image contains 3 ground truth bicycles. A model predicts 4 bounding boxes with confidence scores: 0.95, 0.85, 0.70, 0.40. After IoU matching: Box 1 correct, Box 2 incorrect, Box 3 correct, Box 4 correct.

Step 1 – Cumulative TP and FP:

- After Box 1 (0.95, correct): TP=1, FP=0 → Precision=1.0, Recall=1/3=0.33
- After Box 2 (0.85, incorrect): TP=1, FP=1 → Precision=0.5, Recall=0.33
- After Box 3 (0.70, correct): TP=2, FP=1 → Precision=0.67, Recall=0.66
- After Box 4 (0.40, correct): TP=3, FP=1 → Precision=0.75, Recall=1.0

Step 2 – AP:

$$AP = \frac{1.0 + 0.5 + 0.67 + 0.75}{4} = 0.73$$

Step 3 – mAP: If AP for another class (bus) = 0.65:

$$mAP = \frac{0.73 + 0.65}{2} = 0.69$$

Answer: mAP = **0.69** (69%).

3.7 Non-Maximum Suppression (NMS)

Object detection models often predict multiple bounding boxes for the same object. NMS removes duplicates by:

1. Selecting the box with the highest confidence score.
2. Removing all other boxes of the same class with $\text{IoU} > \text{threshold}$ (typically 0.5) with the selected box.
3. Repeating until no boxes remain.

When NOT to apply NMS: If two overlapping bounding boxes refer to **different objects**, they should not be removed. In such cases, use **Soft-NMS** to reduce confidence instead of removing boxes.

Example: Two boxes both detect a car with $\text{IoU} = 0.7$ and confidence scores 0.9 and 0.6. NMS keeps the 0.9 box and removes the 0.6 box.

3.8 Loss Functions for Object Detection

3.8.1 Contrastive Loss

Used in Siamese networks to learn embeddings where similar objects are close together and dissimilar objects are far apart:

$$L = \frac{1}{2}(1 - Y)D^2 + \frac{1}{2}Y \max(0, m - D)^2$$

where $Y = 1$ if same class, $Y = 0$ if different class, D = distance between embeddings, m = margin.

3.8.2 Triplet Loss

Used to learn discriminative features by comparing an anchor, a positive (same class), and a negative (different class):

$$L = \max(0, D(a, p)^2 - D(a, n)^2 + \alpha)$$

where a = anchor, p = positive, n = negative, α = margin.

Worked Example 11: Triplet Loss Calculation

Given: Anchor embedding $a = [1, 0]$, Positive $p = [0.9, 0.1]$, Negative $n = [0.1, 0.8]$, Margin $\alpha = 0.2$.

Step 1: Compute Euclidean distances:

$$D(a, p) = \sqrt{(1 - 0.9)^2 + (0 - 0.1)^2} = \sqrt{0.01 + 0.01} = \sqrt{0.02} \approx 0.141$$

$$D(a, n) = \sqrt{(1 - 0.1)^2 + (0 - 0.8)^2} = \sqrt{0.81 + 0.64} = \sqrt{1.45} \approx 1.204$$

Step 2: Compute triplet loss:

$$L = \max(0, 0.141^2 - 1.204^2 + 0.2) = \max(0, 0.02 - 1.45 + 0.2) = \max(0, -1.23) = 0$$

Interpretation: Loss is 0 because the anchor is already much closer to the positive than to the negative. The model does not need to update.

4 Module 4: Generative Adversarial Networks (GANs)

4.1 Introduction to GANs

A **Generative Adversarial Network (GAN)** is a deep learning architecture consisting of two competing neural networks: a **Generator** and a **Discriminator**. Invented by Ian Goodfellow in 2014.

Analogy: Like a forger (Generator) creating fake currency and a detective (Discriminator) trying to catch the fakes. Through competition, both improve until the fakes are indistinguishable from reality.

4.2 Core Architecture and Process

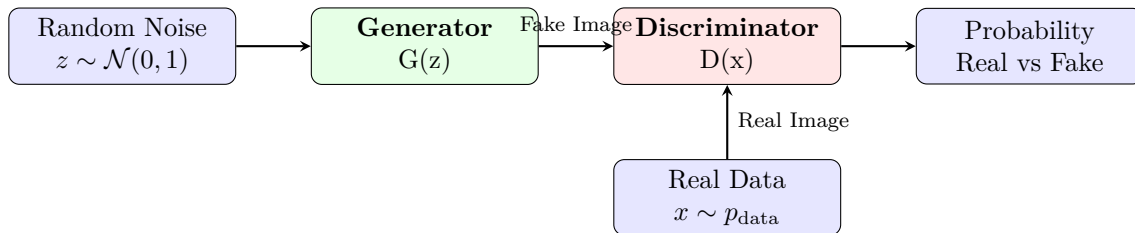


Figure 6: GAN Architecture: Generator takes random noise and produces fake data; Discriminator evaluates both real and fake data, outputting a probability score.

- **Generator (G):**
 - **Input:** Random noise (latent vector).
 - **Output:** Fake data (e.g., generated images).
 - **Goal:** Produce data so realistic that it fools the Discriminator.
- **Discriminator (D):**
 - **Input:** Real data (from the training set) and Fake data (from the Generator).
 - **Output:** Probability score (0 = Fake, 1 = Real).
 - **Goal:** Accurately distinguish between real and fake data.

Training Loop (Min-Max Loss)

1. **Noise Sampling:** Generator takes a random noise vector.
2. **Data Generation:** Generator creates a fake image.
3. **Discrimination:** Discriminator evaluates the fake image against a real image.
4. **Adversarial Training:** Feedback (loss) is sent via backpropagation. The Generator updates weights to minimize the discriminator's accuracy, while the Discriminator updates weights to maximize its accuracy.

Min-Max Objective Function:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\ln D(x)] + \mathbb{E}_{z \sim p_z} [\ln(1 - D(G(z)))]$$

4.3 The Role of the Noise Vector

Why use random noise as input to the Generator?

- **Generate Variety (Diversity):** Without noise, the generator would produce the exact same output every time. Noise acts as a seed for variation.

- **Learn Data Distribution:** The noise maps a simple distribution (like Gaussian) into a complex, high-dimensional real-data distribution.
- **No Bias:** Noise lacks predefined structures, forcing the model to learn pure data patterns.

4.4 Implicit vs. Explicit Density Estimation

- **Implicit Density (GANs):** Learns to generate data that *resembles* the real data distribution without ever explicitly calculating the exact probability distribution.
- **Explicit Density (VAEs - Variational Autoencoders):** Explicitly learns and computes the probability distribution of data using likelihood (like following an exact recipe).

4.5 Limitations of Vanilla GANs

1. **Mode Collapse:** The Generator finds a single output (or a small set of outputs) that easily fools the Discriminator and stops producing diverse samples. (e.g., generating the exact same face every time).
2. **Training Instability:** If the Discriminator gets too strong too quickly, the Generator receives weak gradients and stops learning. If the Generator is too strong, the Discriminator fails.
3. **Vanishing Gradients:** Saturation in the Discriminator's sigmoid activation function can halt the Generator's learning. (Solved partially using Leaky ReLU or different loss functions).
4. **Difficult Hyperparameter Tuning:** Highly sensitive to learning rates, batch sizes, and architecture.

4.6 Advanced GAN Variants

- **CGAN (Conditional GAN):** Takes an additional *label* as input. The label dictates the *category* (e.g., “rose”), while the noise vector dictates the *variation* (shape, color).
- **DCGAN (Deep Convolutional GAN):** Replaces fully connected layers with Convolutional Neural Networks (CNNs). Uses Strided Convolutions (instead of pooling), Batch Normalization, and Leaky ReLU.
- **WGAN (Wasserstein GAN):** Replaces standard divergence loss with the Wasserstein distance (Earth Mover's Distance) to measure differences between real/fake, drastically improving training stability.
- **ProGAN (Progressive GAN):** Starts generating low-resolution images and progressively adds layers to increase resolution and detail.
- **StyleGAN:** Allows fine-grained control over the generated image's style (e.g., pose, lighting, texture).

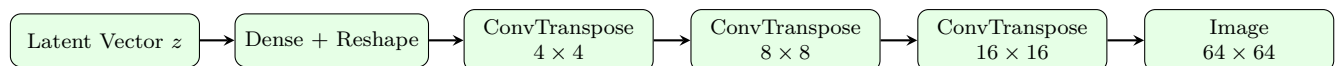


Figure 7: DCGAN Generator Architecture: Latent vector progressively upsampled through transposed convolution layers to generate an image.

4.7 Solved Numericals: Module 4

Worked Example 12: Discriminator Loss Calculation

Given:

- $D(x) = 0.9$ (Discriminator's probability output for a real image)
- $D(G(z)) = 0.3$ (Discriminator's probability output for a fake image)

Objective: Calculate the Discriminator's Loss.

Steps:

1. **Formula:** The Discriminator Loss (using natural logarithm) is defined as:

$$\text{Loss}_D = -[\ln(D(x)) + \ln(1 - D(G(z)))]$$

2. **Substitute values:**

$$\text{Loss}_D = -[\ln(0.9) + \ln(1 - 0.3)]$$

$$\text{Loss}_D = -[\ln(0.9) + \ln(0.7)]$$

3. **Calculate logarithms:**

$$\ln(0.9) \approx -0.105, \quad \ln(0.7) \approx -0.357$$

4. **Final calculation:**

$$\text{Loss}_D = -(-0.105 - 0.357) = -(-0.462) = 0.462$$

Interpretation: A lower loss indicates the Discriminator is doing well. 0.462 is relatively low, meaning it correctly identified the real image (0.9 is close to 1) and the fake image (0.3 is close to 0).

Worked Example 13: KL and JS Divergence

Given:

- Real Distribution $P = [0.5, 0.3, 0.2]$
- Predicted Distribution $Q = [0.2, 0.2, 0.6]$

Objective: Compute Kullback-Leibler (KL) Divergence and Jensen-Shannon Divergence (JSD).

Part 1: KL Divergence ($D_{KL}(P||M)$ and $D_{KL}(Q||M)$)

1. Find average distribution M :

$$M = \frac{1}{2}(P + Q) = \left[\frac{0.5 + 0.2}{2}, \frac{0.3 + 0.2}{2}, \frac{0.2 + 0.6}{2} \right] = [0.35, 0.25, 0.4]$$

2. Compute $D_{KL}(P||M)$:

$$D_{KL}(P||M) = \sum P(x) \ln \left(\frac{P(x)}{M(x)} \right)$$

- $0.5 \ln(0.5/0.35) = 0.5 \ln(1.428) = 0.178$
- $0.3 \ln(0.3/0.25) = 0.3 \ln(1.2) = 0.0546$
- $0.2 \ln(0.2/0.4) = 0.2 \ln(0.5) = -0.1386$

$$D_{KL}(P||M) = 0.178 + 0.0546 - 0.1386 = 0.094$$

3. Compute $D_{KL}(Q||M)$:

$$D_{KL}(Q||M) = \sum Q(x) \ln \left(\frac{Q(x)}{M(x)} \right)$$

- $0.2 \ln(0.2/0.35) = -0.112$
- $0.2 \ln(0.2/0.25) = -0.0446$
- $0.6 \ln(0.6/0.4) = 0.243$

$$D_{KL}(Q||M) = -0.112 - 0.0446 + 0.243 = 0.0864$$

Part 2: Jensen-Shannon Divergence (JSD)

1. Formula:

$$\text{JSD}(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M)$$

2. Calculate:

$$\text{JSD}(P||Q) = \frac{1}{2}(0.094 + 0.0864) = \frac{0.1804}{2} = 0.0902$$

Interpretation: JSD is symmetric and always finite ($0 \leq \text{JSD} \leq 1$ with base 2). Here, $\text{JSD} \approx 0.0902$, indicating a low-to-moderate difference between the distributions.

5 Module 5: Object Segmentation

5.1 Introduction to Segmentation

Image segmentation is the process of grouping pixels with similar visual characteristics into meaningful segments. It is essentially clustering in the spatial domain.

Resolution & DPI: 1024×768 resolution = 786,432 pixels. DPI (Dots Per Inch) determines physical print size/quality, while PPI (Pixels Per Inch) is for screens.

5.2 Types of Segmentation

- **Semantic Segmentation:** Classifies every pixel into a category (e.g., all cars get the same “car” label). Cannot differentiate between two distinct cars next to each other.
- **Instance Segmentation:** Detects individual object instances. If there are two cars, they are labeled as “Car 1” and “Car 2”.
- **Panoptic Segmentation:** Combines both. Uses the concepts of **Things** (countable objects with boundaries, like cars and people) and **Stuff** (uncountable regions without strict boundaries, like sky, grass, road).

Things vs Stuff in Panoptic Segmentation:

- **Things:** Countable and distinct object instances with well-defined boundaries (cars, people, animals, furniture).
- **Stuff:** Uncountable regions without well-defined boundaries (sky, road, grass, walls).

5.3 Convolutional Receptive Fields and Pyramids

- **Receptive Field:** The area of the input image that a neuron “sees”. Shallow layers see small patches (high resolution, edge details). Deep layers see large regions (low resolution, strong semantic meaning).
- **Image Pyramid:** Manually resizing the image multiple times to detect objects at different scales (computationally expensive).

5.4 Key Segmentation Architectures

5.4.1 FPN (Feature Pyramid Network)

Solves multi-scale detection without multiple image passes.

- **Bottom-Up Pathway:** Normal CNN extracting features (spatial resolution drops, semantic meaning increases).
- **Top-Down Pathway:** Upsamples deep, low-resolution layers.
- **Lateral Connections:** Combines the upsampled deep layers with corresponding shallow layers using 1×1 convolutions. Merges semantic strength with spatial detail.

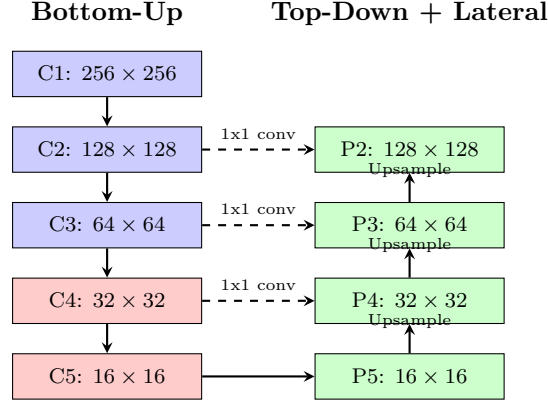


Figure 8: FPN Architecture: Bottom-up pathway extracts features at multiple scales; top-down pathway upsamples deep features and merges with shallow features via lateral connections (1x1 convolutions).

5.4.2 PSPNet (Pyramid Scene Parsing Network)

Uses a **Pyramid Pooling Module** that divides the feature map into different grid sizes (1×1 , 2×2 , 3×3 , 6×6) to capture local and global contexts simultaneously. These are upsampled and concatenated before final pixel-wise softmax prediction.

5.4.3 U-Net (Widely used in Medical Imaging)

- **Contracting Path (Encoder):** Uses convolutions and pooling to capture context (reduces spatial size).
- **Expansive Path (Decoder):** Uses transposed convolutions to upsample and restore spatial resolution.
- **Skip Connections:** Crucial feature. Bypasses the deeper layers to directly pass high-resolution spatial details (like edges/boundaries) from the encoder to the decoder, preventing the loss of localization accuracy.

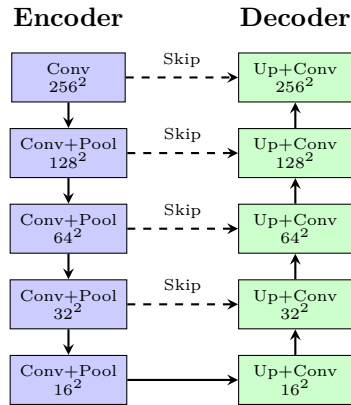


Figure 9: U-Net Architecture: Encoder captures context via pooling; decoder restores resolution via upsampling. Skip connections (dashed) preserve spatial detail from shallow layers to corresponding decoder layers.

5.5 Clustering Methods for Segmentation

- **K-Means:** Groups pixels into K predefined clusters based on similarity (minimizing variance).

- **Mean Shift:** Does not require predefined K . Finds clusters by moving data points towards areas of highest density.
- **DBSCAN:** Density-based. Groups closely packed pixels and isolates outliers/noise. Good for irregular shapes.
- **Agglomerative Hierarchical:** Builds a hierarchy by merging smaller clusters into larger ones. Relies on **Linkage Criteria:** Single (shortest distance), Complete (longest distance), Average, or Centroid.

5.6 Distance Metrics

- **Euclidean:** $d = \sqrt{\sum (x_i - y_i)^2}$ — Straight-line distance (Pythagorean).
- **Manhattan:** $d = \sum |x_i - y_i|$ — Grid-like distance (sum of absolute differences).
- **Cosine Similarity:** $\cos(\theta) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$ — Measures the angle between vectors (ignores magnitude).
- **Minkowski:** Generalized distance formula containing Euclidean and Manhattan.
- **Jaccard Index / Sorensen-Dice:** Measures overlap/similarity between two sets.

5.7 Solved Numericals: Module 5

Worked Example 14: Bilinear Interpolation (Upsampling)

Given: A low-resolution 2×2 grid with the following corner pixel values that we wish to upsample into a 4×4 grid:

- Top-left: 10 Top-right: 20
- Bottom-left: 30 Bottom-right: 40

Objective: Estimate the inner pixel values using bilinear interpolation.

Steps:

1. **Horizontal Interpolation (Rows):** We have 3 intervals to span between the edges.
 - Top Row (10 to 20): Step size = $(20 - 10)/3 = 3.33$.
 - Values: $10 \rightarrow 13.3 \rightarrow 16.7 \rightarrow 20$.
 - Bottom Row (30 to 40): Step size = $(40 - 30)/3 = 3.33$.
 - Values: $30 \rightarrow 33.3 \rightarrow 36.7 \rightarrow 40$.
2. **Vertical Interpolation (Columns):** Now interpolate the middle columns vertically. The step size is $(Bottom - Top)/3$.
 - Column 2 (13.3 to 33.3): Step size = $(33.3 - 13.3)/3 = 6.67$.
 - Values: $13.3 \rightarrow 20.0 \rightarrow 26.7 \rightarrow 33.3$.
 - Column 3 (16.7 to 36.7): Step size = $(36.7 - 16.7)/3 = 6.67$.
 - Values: $16.7 \rightarrow 23.3 \rightarrow 30.0 \rightarrow 36.7$.

Interpretation: The resulting 4×4 interpolated image matrix is smoothly transitioned:

10.0	13.3	16.7	20.0
16.7	20.0	23.3	26.7
23.3	26.7	30.0	33.3
30.0	33.3	36.7	40.0

Worked Example 15: IoU and Dice Coefficient

Given: In an image segmentation task, we have the Ground Truth mask (A) and the Model's Predicted mask (B):

- Ground Truth area ($|A|$) = 50 pixels
- Predicted area ($|B|$) = 60 pixels
- Intersection (correctly predicted pixels, $|A \cap B|$) = 40 pixels

Objective: Calculate the Intersection over Union (IoU) and the Dice Coefficient.

Steps:

1. **Intersection over Union (IoU):**

$$\text{Union } |A \cup B| = |A| + |B| - |A \cap B|$$

$$|A \cup B| = 50 + 60 - 40 = 70 \text{ pixels}$$

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{40}{70} \approx 0.571 \text{ (or 57.1\%)}$$

2. **Dice Coefficient (F1 Score):**

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}$$

$$\text{Dice} = \frac{2 \times 40}{50 + 60} = \frac{80}{110} \approx 0.727 \text{ (or 72.7\%)}$$

Interpretation: Both metrics evaluate segmentation accuracy. IoU penalizes false positives and false negatives more heavily than Dice, which is why IoU (0.57) is lower than Dice (0.72).

6 Module 6: Action Recognition and Object Tracking

6.1 Video Understanding Basics

- **Video vs. Image:** An image has only spatial information (width, height). A video adds a **temporal** dimension (x, y, t) .
- **Why Temporal Matters:** A single frame of a person with hands together could be praying, clapping, or resting. A sequence of frames defines the action.
- **Action Recognition:** “What is happening?” (e.g., running, fighting).
- **Object Tracking:** “Where is it happening over time?” (Locating an object continuously across frames).

Real-world pipeline: Detection $\rightarrow$ Tracking $\rightarrow$ Action Recognition $\rightarrow$ Decision.

6.2 Motion Capture and Modeling

- **Marker-based:** Uses reflective markers on joints and infrared cameras. Highly accurate, used in biomechanics and animation.
- **Markerless:** Uses AI/Pose-estimation (e.g., OpenPose, MoveNet) to detect joints from standard video. Non-invasive.
- **Kinematic Models (How things move, ignoring force):**
 1. **Point Mass Model:** Treats object as a single point. Ignores shape, size, and rotation.
 2. **Rigid Body Model:** Accounts for shape and rotation, assuming the object doesn't deform.
 3. **Articulated Body Model:** Multiple rigid bodies connected by joints (e.g., a human body or robotic arm).
- **Dynamic Models:** Examines *why* things move, analyzing mass, forces, and torques.

6.3 Optical Flow

Definition: The pattern of apparent motion of image objects between two consecutive frames caused by the movement of object or camera.

Brightness Constancy Assumption: The intensity (color/brightness) of a pixel representing a moving object remains constant between frames; only its position (x, y) changes.

$$I_x u + I_y v + I_t = 0$$

where I_x, I_y are spatial gradients, I_t is temporal gradient, and (u, v) is the optical flow vector.

The Aperture Problem: Viewing motion through a small region makes it impossible to determine the true direction of motion along an edge. Therefore, a single pixel is not enough to calculate optical flow (we have 1 equation but 2 unknowns: u and v).

Solutions:

- **Lucas-Kanade Method:** Assumes *Local Motion Consistency* (nearby pixels in a small window move in the same way). Produces **Sparse Optical Flow** (only tracks key feature points/corners).
- **Horn-Schunck Method:** Assumes *Global Smoothness* (motion changes smoothly across the entire image). Produces **Dense Optical Flow** (calculates motion vectors for every single pixel).

6.4 Tracking Methods

- **Kalman Filter:** A predictive model that estimates an object's future position based on its past trajectory and updates the prediction using new measurements. Fast and efficient for linear motion.
 - **Prediction:** Estimate next state based on current state and motion model.
 - **Update:** Correct prediction using actual measurement.
 - Works best when motion is predictable (linear/Gaussian).
- **Particle Filter:** Used for complex, non-linear motion tracking. Represents the state distribution with a set of weighted particles.

Case Study: CCTV Surveillance System

Scenario: A camera monitors a shopping mall to ensure safety and security.

Q1: How will the system follow a person across frames?

- **Detect:** Identify the person in the initial frame using object detection.
- **Track:** Estimate the person's position in subsequent frames based on appearance and motion models.
- **Update:** Continuously update the person's location vector as they move.

Q2: How will the system detect suspicious behavior (e.g., fighting)?

- **Analyze:** Monitor the sequence of spatio-temporal movements across consecutive frames.
- **Identify:** Detect unusual or abnormal motion patterns that deviate from normal walking behavior.
- **Classify:** Use an action recognition classifier to tag the abnormal movement sequence as "fighting".

6.5 3D Vision and Depth Estimation

- **Stereo Vision:** Uses two cameras (Left and Right) to estimate depth, mimicking human eyes.
- **Stereo Matching:** Finding the exact same physical point in both the left and right images.
- **Disparity:** The difference in the pixel coordinates of the same point in the two images.
- **Depth Calculation:** Depth is **inversely proportional** to disparity.
 - *Large disparity = Object is close.*
 - *Small disparity = Object is far away.*

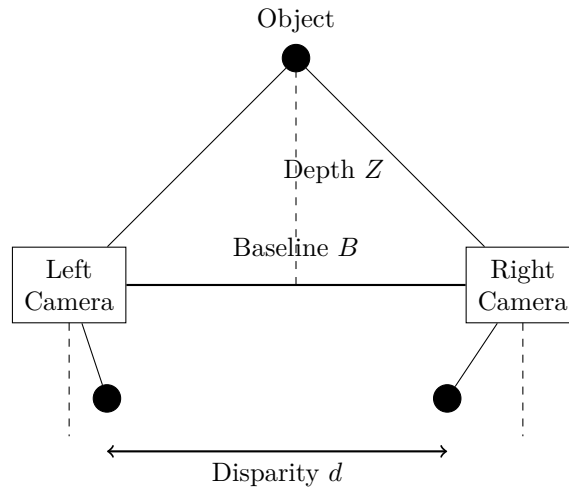


Figure 10: Stereo Vision Triangulation: Two cameras separated by baseline B observe the same object. Disparity $d = x_L - x_R$ is inversely proportional to depth Z .

6.6 Solved Numericals: Module 6

Worked Example 16: Optical Flow (Brightness Constancy)

Given: A tracking system observes a pixel moving between two consecutive frames. The image gradients at the pixel are calculated as follows:

- Spatial gradient in X-direction (I_x) = 2
- Spatial gradient in Y-direction (I_y) = 3
- Temporal gradient (I_t) = -12

It is known that the object is moving **purely horizontally** (meaning the vertical velocity component $v = 0$).

Objective: Find the horizontal velocity u of the pixel.

Steps:

1. **Formula:** State the Brightness Constancy Equation:

$$I_x u + I_y v + I_t = 0$$

2. **Substitute values:**

$$(2)u + (3)(0) + (-12) = 0$$

3. **Solve for u :**

$$2u - 12 = 0 \implies 2u = 12 \implies u = 6$$

Interpretation: The optical flow vector is $(u, v) = (6, 0)$. This means the pixel moved 6 pixels to the right between the two frames.

Worked Example 17: Depth Estimation using Stereo Vision

Given: A stereo vision system has two parallel cameras:

- Focal length (f) = 50 mm
- Baseline distance between cameras (B) = 120 mm
- An object is detected at $x_L = 400$ in the left image and $x_R = 390$ in the right image (in pixels).
- Sensor pixel size = 0.01 mm/pixel

Objective: Calculate the depth (Z) of the object from the cameras.

Steps:

1. **Calculate Disparity in pixels:**

$$d_{\text{pixels}} = x_L - x_R = 400 - 390 = 10 \text{ pixels}$$

2. **Convert Disparity to Physical Units (mm):**

$$d = d_{\text{pixels}} \times \text{pixel size} = 10 \times 0.01 = 0.1 \text{ mm}$$

3. **Calculate Depth using Triangulation Formula:**

$$Z = \frac{f \cdot B}{d}$$

$$Z = \frac{50 \text{ mm} \times 120 \text{ mm}}{0.1 \text{ mm}}$$

$$Z = \frac{6000}{0.1} = 60,000 \text{ mm}$$

4. **Convert to standard units:**

$$60,000 \text{ mm} = 60 \text{ meters}$$

Interpretation: The object is 60 meters away from the stereo camera setup. This demonstrates the inverse relationship: a small disparity (0.1 mm) corresponds to a large depth (60 m).

Appendix: Quick Reference Tables

Numerical Inventory

Module	Worked Example	Concept
1	WE 1	Image Memory Calculation
1	WE 2	Sampling and Quantization
1	WE 3	DCT Coefficient Energy Compaction
1	WE 4	Morphological Opening and Closing
2	WE 5	Chain Code Extraction
2	WE 6	Iterative Global Thresholding
2	WE 7	Otsu's Method
2	WE 8	Hough Transform Line Detection
3	WE 9	Two-Stage vs One-Stage Comparison
3	WE 10	IoU, Precision, Recall, and mAP
3	WE 11	Triplet Loss Calculation
4	WE 12	Discriminator Loss Calculation
4	WE 13	KL and JS Divergence
5	WE 14	Bilinear Interpolation
5	WE 15	IoU and Dice Coefficient
6	WE 16	Optical Flow (Brightness Constancy)
6	WE 17	Depth Estimation (Stereo Vision)

TikZ Diagram Inventory

Module	Diagram
1	Image Acquisition Pipeline
1	Morphological Operations Concept
2	8-Directional Freeman Chain Code
3	R-CNN Evolution (R-CNN / Fast R-CNN / Faster R-CNN)
3	YOLO Grid-Based Detection
4	GAN Architecture (Generator + Discriminator)
4	DCGAN Generator Architecture
5	FPN Architecture (Bottom-Up + Top-Down + Lateral)
5	U-Net Encoder-Decoder with Skip Connections
6	Stereo Vision Triangulation